

Databases

Document Databases: CRUD operaties

Inhoud

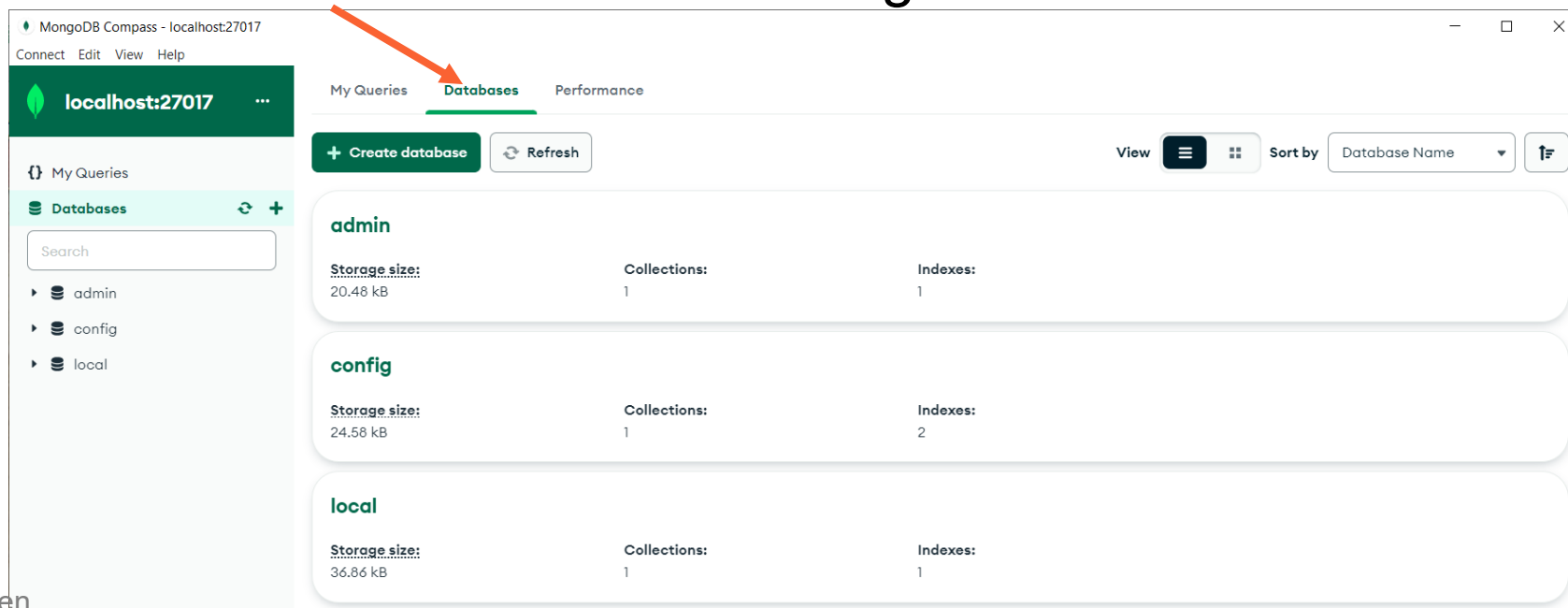
- Gebruik van MongoDB Compass
- Queries in MongoDB
- Create, update en delete in MongoDB

Document databases

Aan de slag met MongoDB

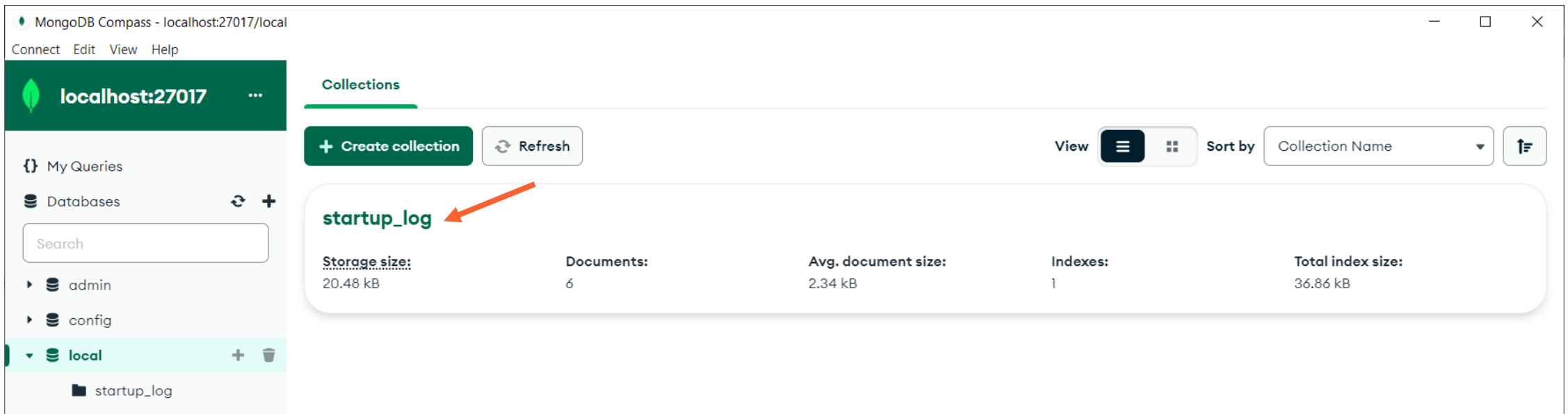
Eerste stappen

- MongoDB Compass bevat drie tabbladen
 - My Queries
 - Databases
 - Performance
- Klik op 'Databases' om de standaard aangemaakte databases te bekijken



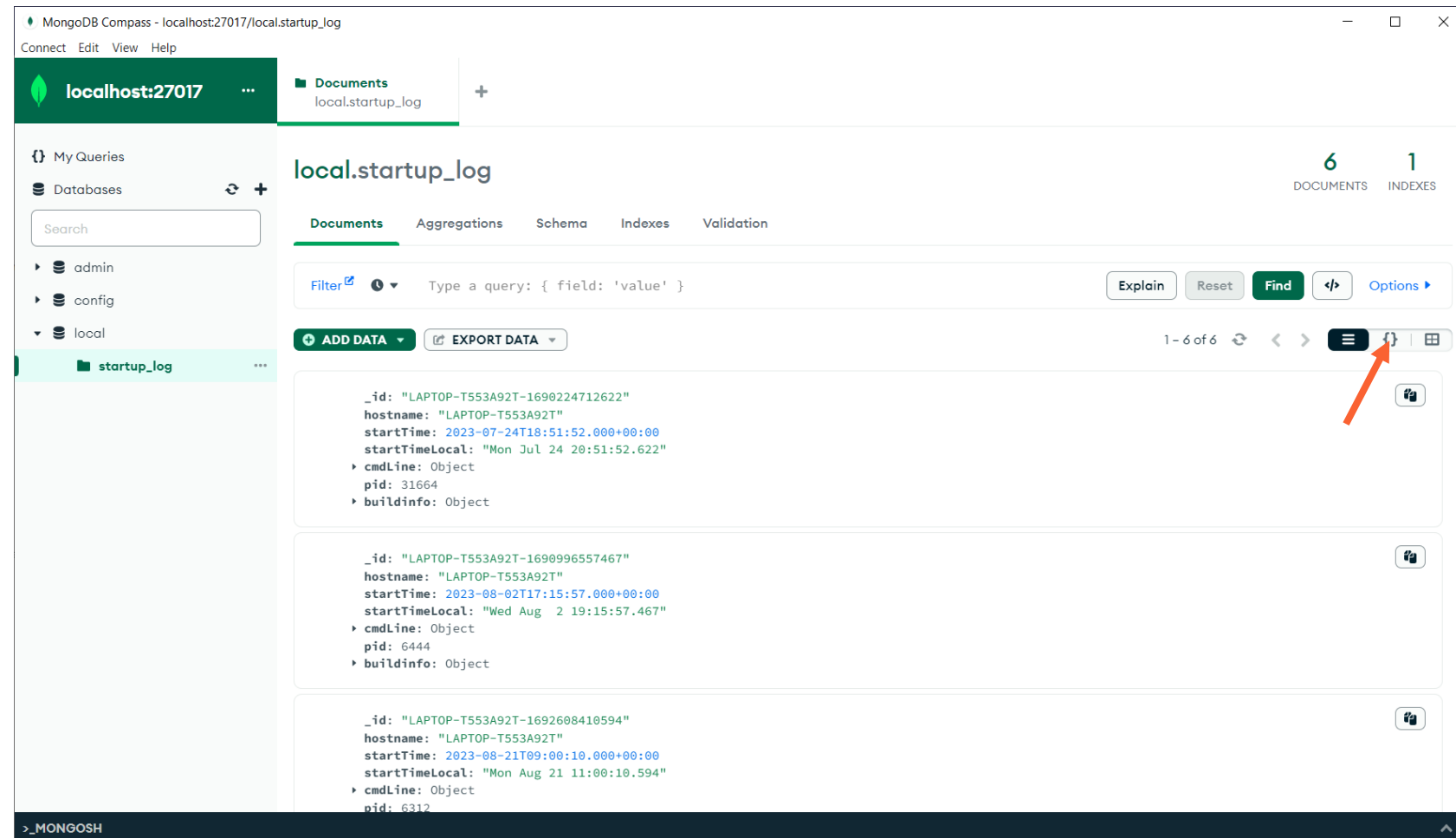
Eerste stappen

- Klik op de 'local' database
 - Je ziet één collectie in deze database: 'startup_log'
 - Het overzicht toont het aantal documenten in de collectie
- Klik op de 'startup_log' collectie om deze te openen



Eerste stappen

- Standaard wordt een document getoond op een overzichtelijke manier
- Via de '{}' knop kan de data bekeken worden in BSON formaat



Eerste stappen

- Telkens dat de MongoDB Community Server opstart, wordt er een document met diagnostische informatie toegevoegd aan de 'startup_log' collectie

ADD DATA EXPORT DATA

1 - 6 of 6

```
{
  "_id": "LAPTOP-T553A92T-1690224712622",
  "hostname": "LAPTOP-T553A92T",
  "startTime": {
    "$date": "2023-07-24T18:51:52.000Z"
  },
  "startTimeLocal": "Mon Jul 24 20:51:52.622",
  "cmdLine": {
    "config": "C:\\Program Files\\MongoDB\\Server\\6.0\\bin\\mongod.cfg",
    "net": {
      "bindIp": "127.0.0.1",
      "port": 27017
    },
    "service": true,
    "storage": {
      "dbPath": "C:\\Program Files\\MongoDB\\Server\\6.0\\data",
      "journal": {
        "enabled": true
      }
    },
    "systemLog": {},
    "pid": {},
    "buildinfo": {}
  }
}
```

ObjectId

- Elk document in MongoDB bevat een ‘_id’ property
 - Bevat een UUID zodat alle documenten uniek geïdentificeerd kunnen worden
 - Gelijkaardig aan een primaire sleutel
 - Een document dat toegevoegd wordt mag, maar moet geen ‘_id’ property bevatten
 - Als de property ontbreekt, wordt deze automatisch gegenereerd
 - Het datatype is ‘ObjectId’, een type dat enkel in BSON beschikbaar is.
 - De waarde is immutable en kan dus niet meer aangepast worden nadat het document toegevoegd is

```
▼ {  
  "_id": "LAPTOP-T553A92T-1690224712622",  
  "hostname": "LAPTOP-T553A92T",  
  ▶ "startTime": {...},  
  "startTimeLocal": "Mon Jul 24 20:51:52.622",  
  ▶ "cmdLine": {...},  
  ▶ "pid": {...},  
  ▶ "buildinfo": {...}  
}
```


Propertynamen

- De volgende restricties gelden voor propertynamen in een document
 - Een propertynaam mag het null-karakter (een onzichtbaar teken) niet bevatten
 - Speciale karakters zijn niet toegestaan, met uitzondering van . en \$
 - Toch gebruik je deze best niet
 - Spaties zijn toegestaan, maar worden best vermeden
- De namen van databases en collecties mogen geen spaties bevatten

Volgorde van properties

Volgorde van velden in queries

- In een BSON document zijn properties geordend in JSON niet
- Voor queries gelden de volgende regels:
 - Wanneer documenten vergeleken worden is de volgorde belangrijk
 - {a: 1, b:1} is gelijk aan {a: 1, b: 1}
 - {a: 1, b:1} is **niet** gelijk aan {b: 1, a: 1}

Volgorde van properties

Volgorde van properties in write operaties

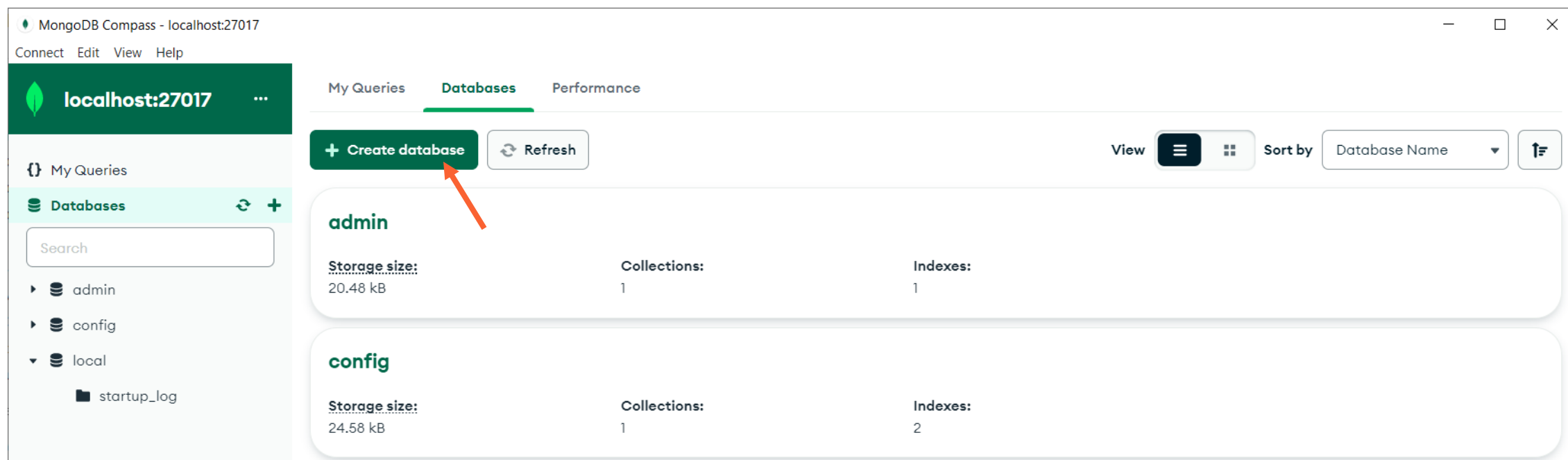
- MongoDB bewaard de volgorde van properties tijdens write operaties, met volgende uitzonderingen:
 - De ‘_id’ property is altijd het eerste veld in een document
 - Updates die velden hernoemen kunnen de volgorde van de velden wijzigen

Document databases

Hands-on: Queries

Database aanmaken

- Een database aanmaken kan via MongoDB Compass (GUI) en via MongoDB Shell (CLI)
- In MongoDB Compass: Ga naar het overzicht van de databases en klik op 'Create Database'



Database aanmaken

- Maak een database met de naam 'coffeeshop' en een collectie 'customers'
 - Elke database moet minstens één collectie bevatten

Create Database ×

Database Name

Collection Name

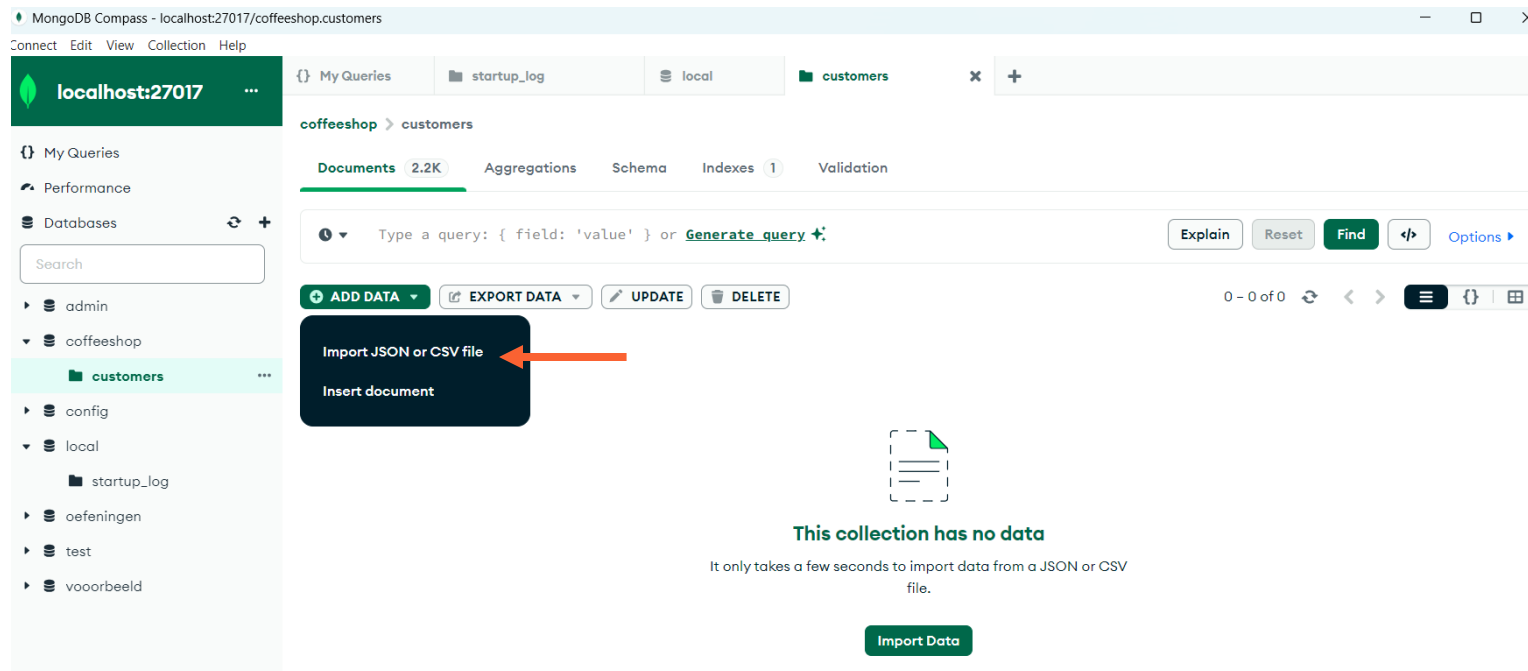
☐ **Time-Series**
Time-series collections efficiently store sequences of measurements over a period of time. [Learn More](#)

➤ **Additional preferences** (e.g. Custom collation, Capped, Clustered collections)

CancelCreate Database

Documenten toevoegen

- Document kan toegevoegd worden door
 - Manueel documenten toe te voegen via de GUI
 - JSON of CSV bestanden te uploaden
- Voeg 'customers.json' toe aan de 'customers' collectie



Documenten toevoegen

- Merk op dat MongoDB automatisch een '_id' property heeft toegevoegd aan elk document
 - Deze zijn gegenereerd tijdens de import en zijn dus anders op elke machine
- De collectie bevat nu 2246 documenten

 ADD DATA  EXPORT DATA

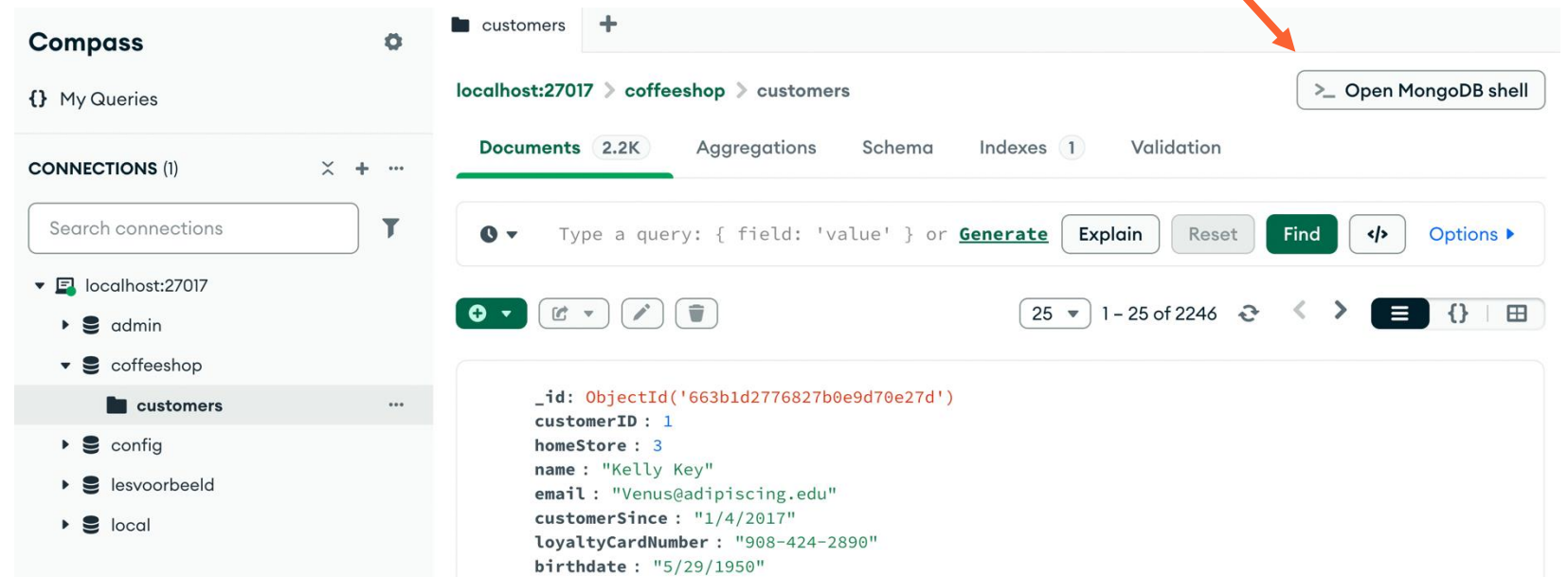
1 - 20 of 2246



```
_id: ObjectId('64f49dfe46aa0bdb0059b345')  
customerID: 1  
homeStore: 3  
name: "Kelly Key"  
email: "Venus@adipiscing.edu"  
customerSince: "1/4/2017"  
loyaltyCardNumber: "908-424-2890"  
birthdate: "5/29/1950"  
gender: "M"  
birthYear: 1950
```


Documenten zoeken

- Eén van de meest voorkomende operaties is het zoeken van documenten (analoog aan een SELECT in SQL)
- We gebruiken de MongoDB Shell om queries aan te maken
- Klik op 'Open MongoDB shell' in de rechter bovenhoek van MongoDB Compass om de shell te openen



Documenten zoeken

- Voordat we data opvragen, moet de database waarin we werken geselecteerd worden
 - *use <database>* (vervang database met de naam van de database)
- Selecteer de *coffeeshop* collectie met het volgende commando
 - *use coffeeshop*

```
>_MONGOSH
> use coffeeshop
< switched to db coffeeshop
coffeeshop>
```

Documenten zoeken

- Via het *find* commando kunnen we documenten ophalen binnen een collectie
 - `db.<collection>.find()`
 - Vergelijkbaar met `SELECT * FROM <schema>.<table>;`
- Via dit commando worden de eerste 20 documenten getoond in de shell
- Gebruik onderstaand commando om de documenten uit de customers collectie op te halen
 - `db.customers.find()`

```
> db.customers.find()
< {
  _id: ObjectId('662b849946ceff41c9b54f89'),
  customerID: 1,
  homeStore: 3,
  name: 'Kelly Key',
  email: 'Venus@adipiscing.edu',
  customerSince: '1/4/2017',
  loyaltyCardNumber: '908-424-2890',
  birthdate: '5/29/1950',
  gender: 'M',
  birthYear: 1950
}
```

Documenten zoeken

- MongoDB geeft standaard 20 documenten terug
- Via het *it* commando kunnen we itereren door de rest van de documenten

```
gender: 'M',
birthYear: 1953
}
{
  _id: ObjectId("64f49dfe46aa0bdb0059b358"),
  customerID: 20,
  homeStore: 3,
  name: 'Nathan Riggs',
  email: 'Iris@aptent.com',
  customerSince: '2/26/2017',
  loyaltyCardNumber: '639-354-7929',
  birthdate: '8/21/1953',
  gender: 'M',
  birthYear: 1953
}
Type "it" for more
```

```
> it
< {
  _id: ObjectId("64f49dfe46aa0bdb0059b359"),
  customerID: 21,
  homeStore: 3,
  name: 'Mufutau Rosa',
  email: 'Haviva@inceptos.gov',
  customerSince: '2/28/2017',
  loyaltyCardNumber: '097-053-2704',
  birthdate: '10/22/1953',
  gender: 'M',
  birthYear: 1953
}
{
  _id: ObjectId("64f49dfe46aa0bdb0059b35a"),
  customerID: 22,
  homeStore: 3,
  name: 'Channing Larson',
```

Documenten zoeken

- Resultaten kunnen gefilterd worden, vergelijkbaar met de WHERE clause in SQL
 - Filters worden toegevoegd als parameter aan de *find* functie
 - Filters worden gestructureerd in een JavaScript object
 - *db.<collection>.find({key: value})*

Documenten zoeken

- Voorbeeld 1: Haal alle klanten op die geboren zijn in 1957
 - *db.customers.find({birthYear: 1957})*

```
> db.customers.find({birthYear: 1957})
< {
  _id: ObjectId('662b849946ceff41c9b54fb0'),
  customerID: 40,
  homeStore: 3,
  name: 'Lucas Copeland',
  email: 'Emi@venenatis.edu',
  customerSince: '4/22/2017',
  loyaltyCardNumber: '180-267-4872',
  birthdate: '1/15/1957',
  gender: 'M',
  birthYear: 1957
}
{
  _id: ObjectId('662b849946ceff41c9b54fb1'),
  customerID: 41,
  homeStore: 3,
  name: 'Harrison Cash',
  email: 'Uriel@imperdiet.us',
  customerSince: '4/25/2017',
  loyaltyCardNumber: '094-153-0253',
  birthdate: '3/18/1957',
  gender: 'M',
  birthYear: 1957
}
```

Documenten zoeken

- Voorbeeld 2: Haal alle klanten op met de naam 'Lesley Benson'
 - *db.customers.find({name: "Lesley Benson"})*

```
> db.customers.find({name: 'Lesley Benson'})
< {
  _id: ObjectId('662b849946ceff41c9b550c7'),
  customerID: 319,
  homeStore: 3,
  name: 'Lesley Benson',
  email: 'Laura@eleifend.net',
  customerSince: '2/10/2017',
  loyaltyCardNumber: '685-365-3310',
  birthdate: '8/29/1952',
  gender: 'F',
  birthYear: 1952
}
```

Documenten zoeken

- Filteren op meerdere kolommen is mogelijk door extra key-value paren toe te voegen aan de parameter van de *find* functie
 - `db.<collection>.find({key1: value1, key2: value2, ..., keyn: valuen})`

Documenten zoeken

- Voorbeeld 3: Haal alle vrouwelijke klanten op die geboren zijn in 1973
 - *db.customers.find({birthYear: 1973, gender: 'F'})*

```
> db.customers.find({birthYear: 1973, gender: 'F'})
< {
  _id: ObjectId('662b849946ceff41c9b55167'),
  customerID: 479,
  homeStore: 3,
  name: 'Glenna Woodward',
  email: 'Demetria@dis.org',
  customerSince: '1/7/2018',
  loyaltyCardNumber: '179-475-7139',
  birthdate: '1/27/1973',
  gender: 'F',
  birthYear: 1973
}
```

Documenten zoeken

- De velden die teruggegeven worden door *find* kunnen gefilterd worden
 - Analooq aan SELECT **<column>** FROM <schema>.<tabel>;
- De tweede parameter van de *find* functie is een object met keys
 - Alle keys die de waarde 1 krijgen, worden getoond
 - Alle keys die de waarde 0 krijgen, worden verborgen
- Als je het aantal properties in het resultaat wilt beperken, maar geen filters wilt toepassen, dan kan je {} gebruiker voor de eerste parameter van *find*
- *db.<collection>.find({key: value}, {key₁: 1, key₂: 1})*

Documenten zoeken

- Voorbeeld 5: Haal alle klanten op die in 1973 geboren zijn en toon hun naam en loyalty card nummer
 - `db.customers.find({birthYear: 1973}, {name: 1, loyaltyCardNumber: 1})`

```
> db.customers.find({birthYear: 1973}, {name: 1, loyaltyCardNumber: 1})
< {
  _id: ObjectId('662b849946ceff41c9b5500e'),
  name: 'Ian Duncan',
  loyaltyCardNumber: '814-963-1003'
}
{
  _id: ObjectId('662b849946ceff41c9b5500f'),
  name: 'Matthew Mcneil',
  loyaltyCardNumber: '942-295-9967'
}
```

Documenten zoeken

- Voorbeeld 6: Haal alle klanten op en toon hun naam en email
 - `db.customers.find({}, {name: 1, email: 1})`

```
> db.customers.find({}, {name: 1, email: 1})
< {
  _id: ObjectId('662b849946ceff41c9b54f89'),
  name: 'Kelly Key',
  email: 'Venus@adipiscing.edu'
}
{
  _id: ObjectId('662b849946ceff41c9b54f8a'),
  name: 'Clark Schroeder',
  email: 'Nora@fames.gov'
}
```

Documenten zoeken

- Voorbeeld 7: Haal alle klanten op, toon hun naam en email en verberg het id
 - `db.customers.find({}, {name: 1, email: 1, _id: 0})`

```
> db.customers.find({}, {name: 1, email: 1, _id: 0})  
< {  
  name: 'Kelly Key',  
  email: 'Venus@adipiscing.edu'  
}  
{  
  name: 'Clark Schroeder',  
  email: 'Nora@fames.gov'  
}
```

Aantal resultaten tellen

- Via de *count* functie kan het aantal documenten die voldoen aan de opgegeven filter opgevraagd worden
 - `db.<collection>.find().count()`
- Voorbeeld 8: Hoe veel documenten zitten er in de *customers* collectie?
 - `db.customers.find().count()`
- Voorbeeld 9: Hoe veel mannelijke klanten zitten er in de *customers* collectie?
 - `db.customers.find({gender: 'M'}).count()`

```
> db.customers.find().count()
< 2246
> db.customers.find({gender: 'M'}).count()
< 726
```

Resultaten sorteren

- MongoDB kan de resultaten sorteren
 - Analooq aan de ORDER BY clause
- De *sort* functie sorteert de uitvoer aan de hand van een JSON parameter waarin de keys vermeld worden waarop gesorteerd moet worden
 - 1: Sorteert oplopend
 - -1: Sorteert aflopend
- *db.<collection>.find().sort({key: 1})*

Resultaten sorteren

- Voorbeeld 10: Toon de namen van alle klanten in alfabetische volgorde
 - `db.customers.find({}, {name: 1, _id: 0}).sort({name: 1})`

```
> db.customers.find({}, {name: 1, _id: 0}).sort({name: 1})
< {
  name: 'Aaron'
}
{
  name: 'Abbot'
}
{
  name: 'Abbot Charles'
}
{
  name: 'Abel Hood'
}
```


Resultaten sorteren

- Voorbeeld 11: Toon de naam en het emailadres van alle klanten, aflopend gesorteerd op email
 - `db.customers.find({}, {name: 1, email: 1, _id: 0}).sort({email: -1})`

```
> db.customers.find({}, {name: 1, email: 1, _id: 0}).sort({email: -1})
< {
  name: 'Kylie',
  email: 'Zoe@eget.edu'
}
{
  name: 'Deirdre',
  email: 'Zia@euismod.net'
}
{
  name: 'Charity',
  email: 'Zeus@velit.us'
}
```

Resultaten sorteren

- Voorbeeld 12: Toon het geboortjaar van alle klanten, met de jongste persoon bovenaan. In het geval er meerdere personen zijn met hetzelfde geboortjaar, sorteer je alfabetisch op naam
 - `db.customers.find({}, {name: 1, birthYear: 1, _id: 0}).sort({birthYear: -1, name: 1})`

```
> db.customers.find({}, {name: 1, birthYear: 1, _id: 0}).sort({birthYear: -1, name: 1})
< {
  name: 'Amethyst',
  birthYear: 2001
}
{
  name: 'Ava Mullins',
  birthYear: 2001
}
{
  name: 'Carter Kelly',
  birthYear: 2001
}
```

Unieke waarden

- Via de *distinct* functie worden de unieke waarden van een veld teruggegeven
 - Niet te combineren met *find*
 - Twee parameters
 - 1: De property waarvan je de unieke waarden wilt ophalen
 - 2: Optioneel, de filters die toegepast moeten worden op de documenten
 - Het resultaat van *distinct* is een *array*
- `db.<collection>.distinct('key', [{key: value}])`

Unieke waarden

- Voorbeeld 12: Toon alle unieke genders van de personen die lid zijn sinds 7/29/2017
 - *db.customers.distinct('gender', {customerSince: '7/29/2017'})*

```
> db.customers.distinct('gender', {customerSince: '7/29/2017'})  
< [ 'F', 'M' ]
```

Document databases

Hands-on: Insert, update & delete

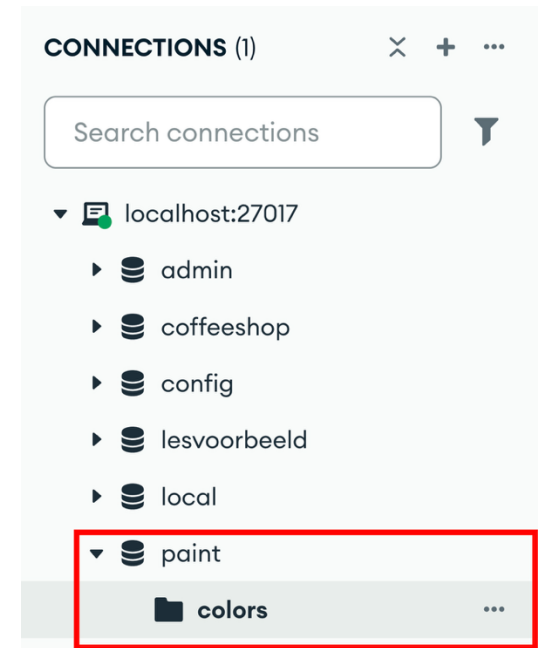
Database aanmaken

- Een database kan aangemaakt worden via MongoDB Compass (GUI) en via MongoDB Shell (CLI)
 - Raadpleeg slide 4 voor GUI instructies
- MongoDB Shell:
 - Gebruik het '*use*' commando om een nieuwe database aan te maken
 - Zodra je een collectie aanmaakt, wordt de nieuwe database ook aangemaakt
- Syntax
 - *use* *<database>* (vervang *<database>* met de naam van de database)

Collectie aanmaken

- In de MongoDB Shell kan een collectie aangemaakt worden met:
 - `db.createCollection('<name>')`
- Voorbeeld 13: Maak een nieuwe collectie 'colors' in de database 'paint'
 - `use paint`
 - `Db.createCollection('colors')`

```
> use paint
< already on db paint
> db.createCollection('colors')
< { ok: 1 }
```



Een document aanmaken

- Een document toevoegen
 - `db.collection.insertOne(<document>)`
 - Het ObjectId wordt door MongoDB aangemaakt en teruggegeven door de `insertOne` methode.
- Voorbeeld 14: Voeg de kleur 'red' toe aan de 'colors' collectie
 - `db.colors.insertOne({name: 'red', hexadecimal: 'FF0000', rgb: '255,0,0'})`

```
> db.colors.insertOne({name: 'red', hexadecimal: 'FF0000', rgb: '255,0,0'})
< {
  acknowledged: true,
  insertedId: ObjectId('679379a9790ea72332b00752')
}
paint> |
```


Verschillende documenten toevoegen

- Meerdere documenten toevoegen
 - *db.collection.insertMany([<document₁>, <document₂>, ... , <document_n>])*

Verschillende documenten toevoegen

- Voorbeeld 15: Voeg de kleur 'green' en 'blue' toe aan de 'colors' collectie
 - `db.colors.insertMany([`
 `{name: 'green', hexadecimal: '00FF00', rgb: '0,255,0'},`
 `{name: 'blue', hexadecimal: '0000FF', rgb: '0,0,255'},`
 `])`

```
> db.colors.insertMany([
  {name: 'green', hexadecimal: '00FF00', rgb: '0,255,0'},
  {name: 'blue', hexadecimal: '0000FF', rgb: '0,0,255'},
])
< {
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('67937af1790ea72332b00753'),
    '1': ObjectId('67937af1790ea72332b00754')
  }
}
paint>
```

Een document updaten

- Het eerste document dat aan een filter voldoet aanpassen
 - `db.collection.updateOne(<filter>, { $set: {<key1>: <value1>, ..., <keyn>: <valuen>}})`
- Voorbeeld 16: Pas de schakering van rood aan
 - `db.colors.updateOne({name: 'red'}, {$set: {hexadecimal: 'C80000', rgb: '200,0,0'}})`

```
> db.colors.updateOne({name: 'red' }, {$set: {hexadecimal: 'C80000', rgb: '200,0,0'}})
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 0,
  upsertedCount: 0
}
```

Meerdere documenten updaten

- Alle documenten die aan een filter voldoen
 - `db.collection.updateMany(<filter>, { $set: {<key1>: <value1>, ..., <keyn>: <valuen>}})`
- Voorbeeld 17: Voeg de kleurruimte toe aan alle documenten
 - `db.colors.updateMany({}, {$set: {colorSpace: 'rgb'}})`

```
> db.colors.updateMany({}, {$set: {colorSpace: 'rgb'}})
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 3,
  modifiedCount: 3,
  upsertedCount: 0
}
```

Document verwijderen

- Het eerste document verwijderen dat voldoet aan een filter
 - `db.collection.deleteOne(<filter>)`
- Voorbeeld 18: Het eerste document met de kleurruimte 'rgb' verwijderen
 - `db.colors.deleteOne({colorSpace: 'rgb'})`

```
> db.colors.deleteOne({colorSpace: 'rgb'})  
< {  
  acknowledged: true,  
  deletedCount: 1  
}  
paint>
```

Meerdere documenten verwijderen

- Alle documenten verwijderen dat voldoet aan een filter
 - *db.collection.deleteMany(<filter>)*
- Voorbeeld 19: Alle documenten met de kleurruimte 'rgb' verwijderen
 - *db.colors.deleteMany({colorSpace: 'rgb'})*

```
> db.colors.deleteMany({colorSpace: 'rgb'})  
< {  
  acknowledged: true,  
  deletedCount: 2  
}  
paint>
```

Collecties & databases verwijderen

- Een collectie verwijderen
 - `db.collection.drop()`
- Verwijder de database waarin je aan het werken bent
 - `db.dropDatabase()`
- Voorbeeld 20: Verwijder de 'colors' collection en de 'paint' database
 - `db.colors.drop()`
 - `db.dropDatabase()`

```
> db.colors.drop()
< true
> db.dropDatabase()
< { ok: 1, dropped: 'paint' }
paint>
```